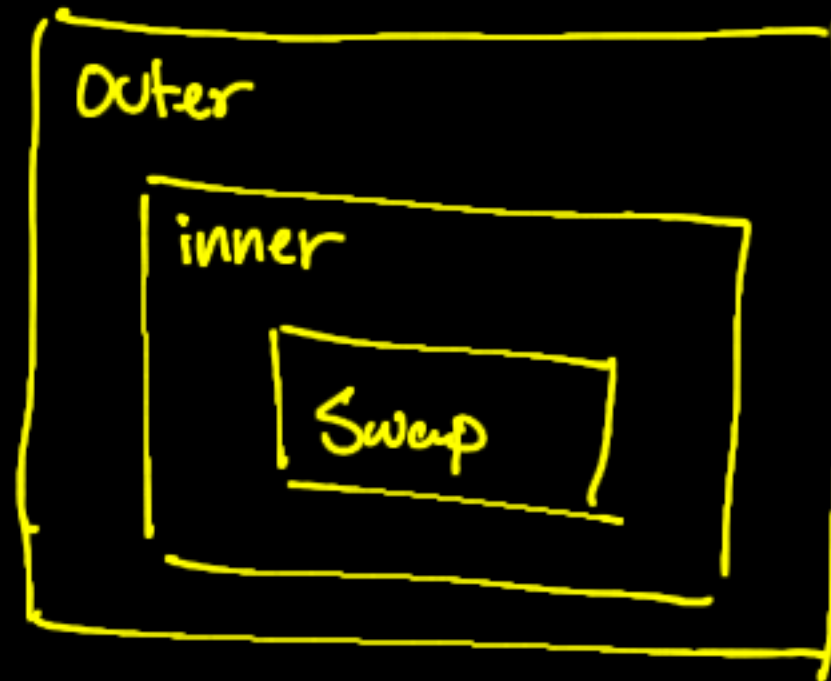


① Bubble sort

29 10 14 37 13
 10 29 14
 10 14 29 37 13
 10 14 29 13 37
 10 14 29
 10 14 13 29 37
 10 13 14 29 37
 10 13 14 29 37

bubblesort (array[I], n)
 for (pass=1; pass < n; pass++)
 for (i=0; i < n-pass; i++)
 if (array[i] > array[i+1])
 temp = array[i]
 array[i] = array[i+1]
 array[i+1] = temp

$O(n^2)$



→

② bubblesort (array[I], n)
 for (pass=1; pass < n; pass++)
 for (i=0; i < n-pass; i++)
 if (array[i] > array[i+1])
 temp = array[i]
 array[i] = array[i+1]
 array[i+1] = temp

← n-1 outer passes
 ← n-i inner passes
 ← n-i comparisons
 ← n-i swaps (3 steps each)

Focus on the comparisons

If n=10 items, then
 pass=1, n-1=9
 + pass=2, n-2=8
 + pass=3, n-3=7
 + pass=4, n-4=6
 + ...
 + pass=9, n-9=1

Thus: 9+8+7+6+...+1=45

This is the summation series
 $Sum = \frac{x(x+1)}{2} = \frac{9(9+1)}{2} = 45$

Therefore, let $x = n-1$
 $Sum = \frac{(n-1)(n-1+1)}{2} = \frac{n^2-n}{2}$

Furthermore the swapping
 (comparison + swap step) = 4
 total = $4 \left(\frac{n^2-n}{2} \right)$

= $2n^2 - 2n$

= $O(n^2)$

Can we do better?

③ bubblesort (array[I], n)

swapped = true
 for (pass=1; pass < n && swapped, pass++)

swapped = false
 for (i=0; i < n-pass; i++)
 if (array[i] > array[i+1])
 temp = array[i]
 array[i] = array[i+1]
 array[i+1] = temp
 swapped = true

Short the algorithm

Does this improve the algorithm?

Think:

Best case...

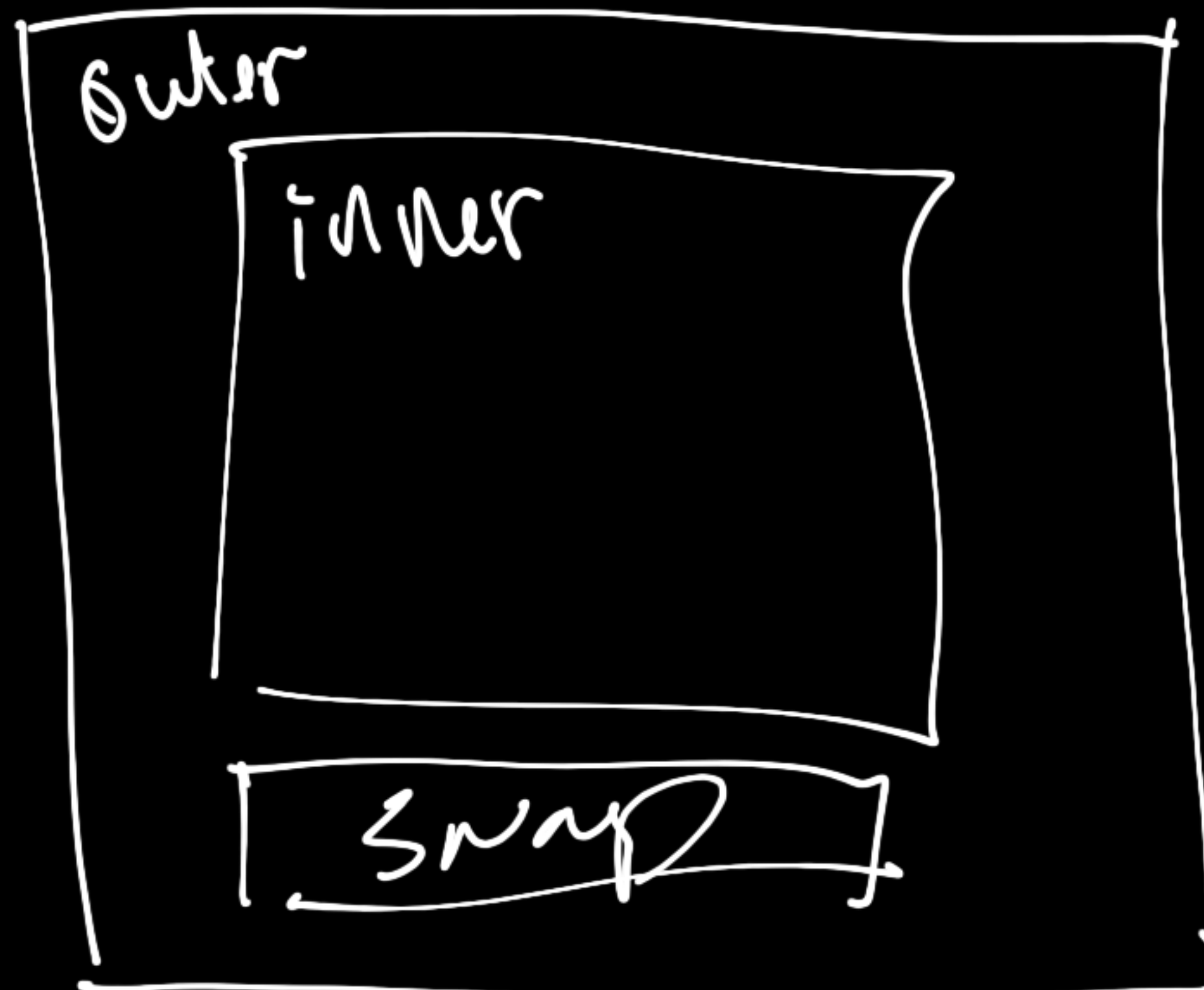
Average case...

Worst case...

↓

④ Selection Sort

29 10 14 37 13
29 10 14 13 | 37
13 10 14 | 29 37
13 10 | 14 29 37
10 | 13 14 29 37



- ① select the largest value
- ② Swap it with the last position of the remaining unsorted array

Performance? $O(n^2)$

for each of the $n-1$ passes

- find largest value
- swap it

5 4 3 2 1
1 4 3 2 5
1 2 3 | 4 5



⑤ Insertion sort

loc = 1 29 10 14 37 13

loc = 0 29 29

loc = 2 10 29 14 37 13

loc = 1 10 14 29 37 13

loc = 3 10 14 29 37 13

loc = 4 10 14 29 37 13

loc = 3 10 13 14 29 37

loc = 2

loc = 1

insertionSort(a[], n)

for unsorted = 1 to n-1

next item = a[unsorted]

loc = unsorted

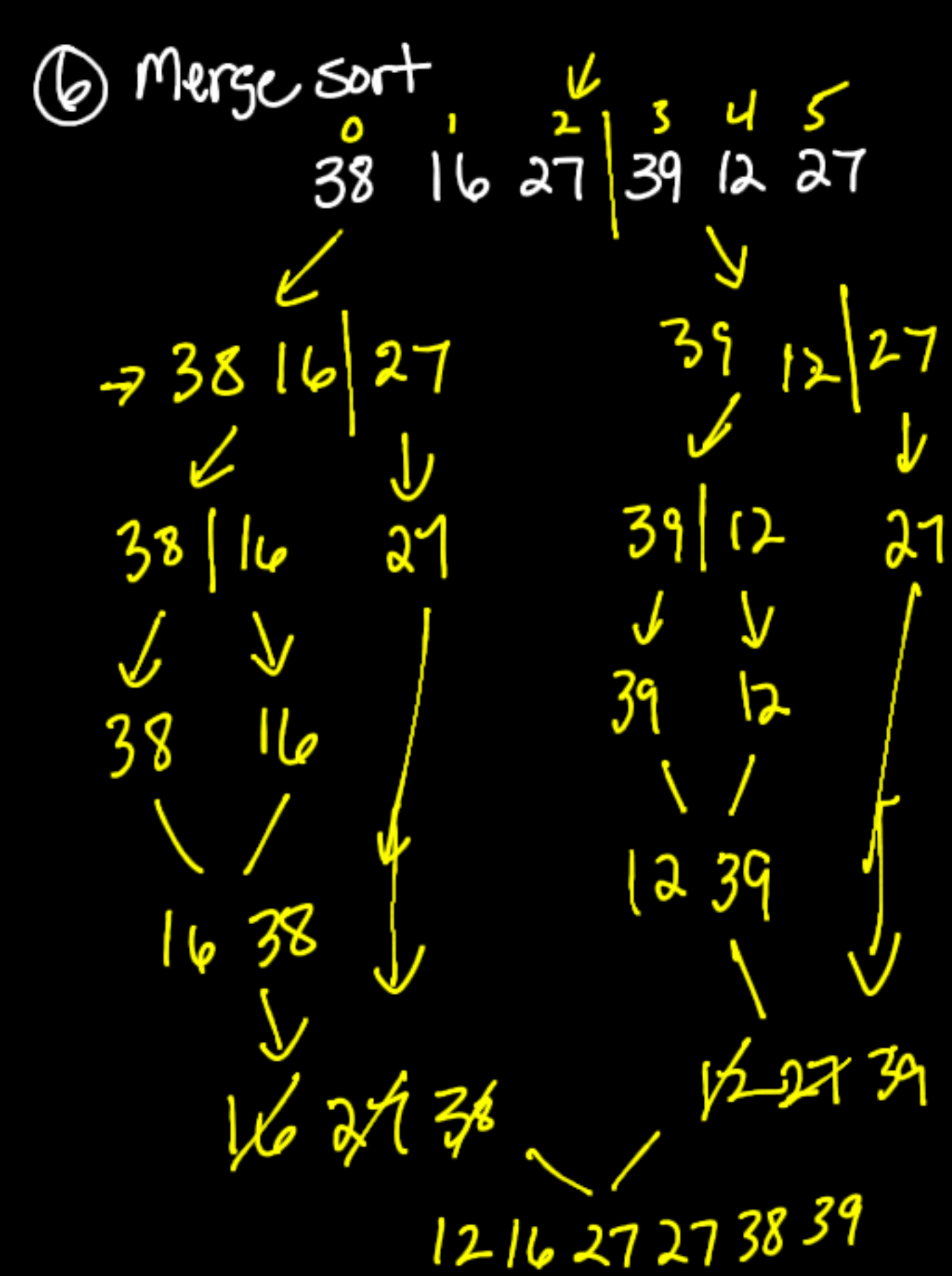
while (loc > 0 && a[loc-1] > next item)

 a[loc] = a[loc-1]

 loc = loc - 1

a[loc] = next item





mergesort(first, last) →

if (first < last)

mid = (first + last) / 2

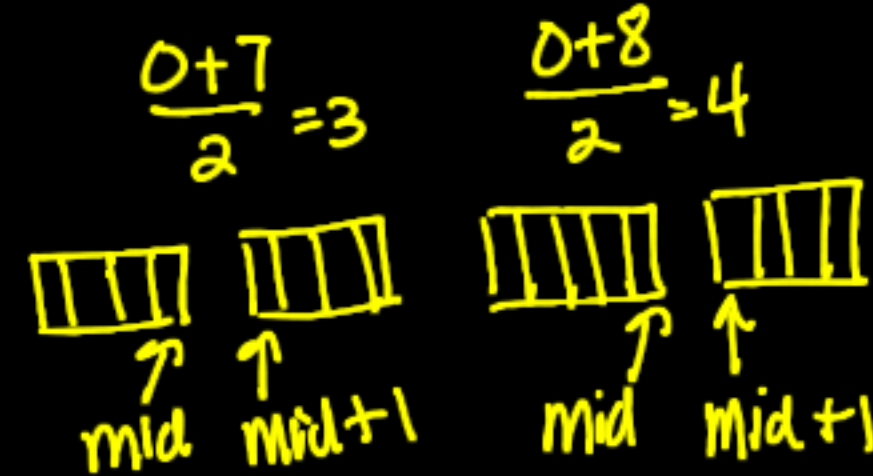
mergesort(first, mid)

mergesort(mid+1, last)

merge(first, mid, last)

see code →

Note: mid is using integer arithmetic



① mergesort(first, last)

if (first < last)

mid = (first + last) / 2

mergesort(first, mid)

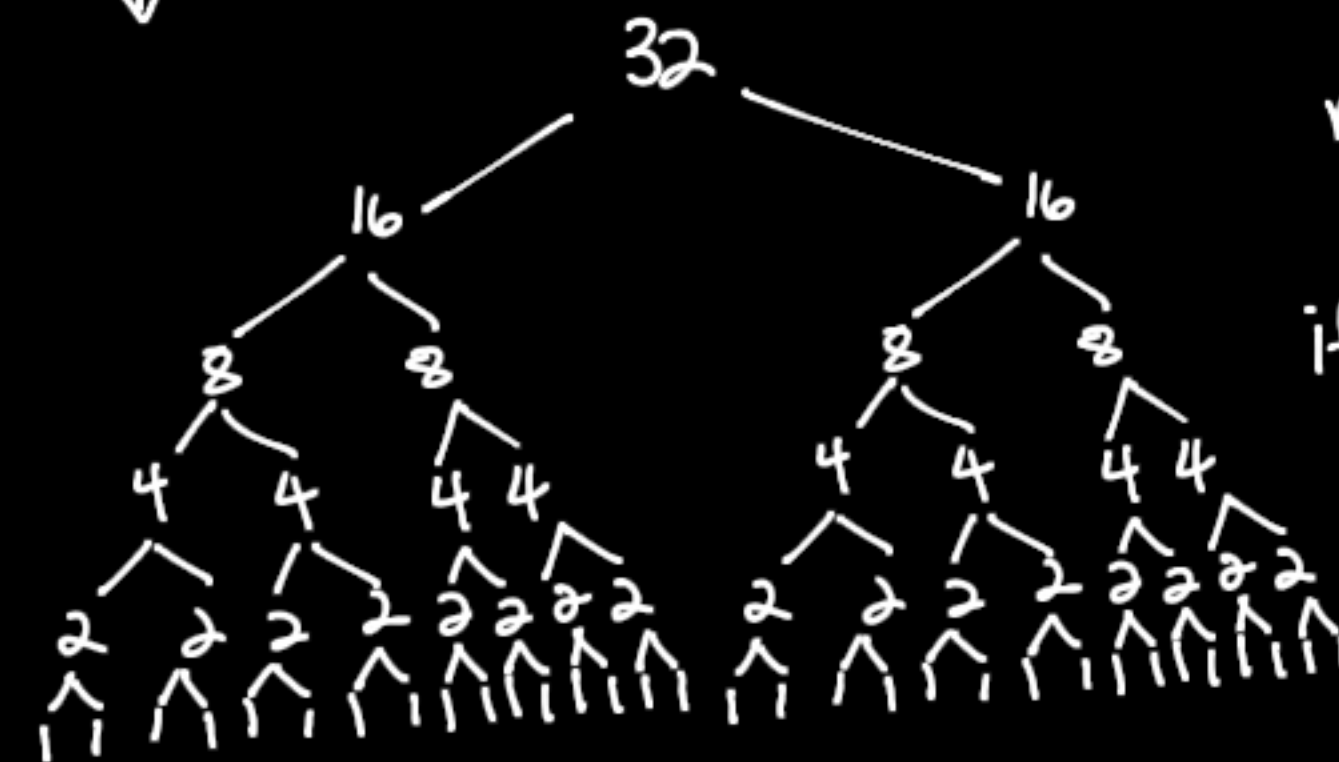
mergesort(mid+1, last)

merge(first, mid, last)

Example n=32

$$k = \log_2 32 = 5$$

h	p	size	
0	1	32	$\frac{n}{2^k} = 1$
1	2	16	
2	4	8	
3	8	4	$\frac{32}{2^5} = 1$
4	16	2	
5	32	1	



Analysis

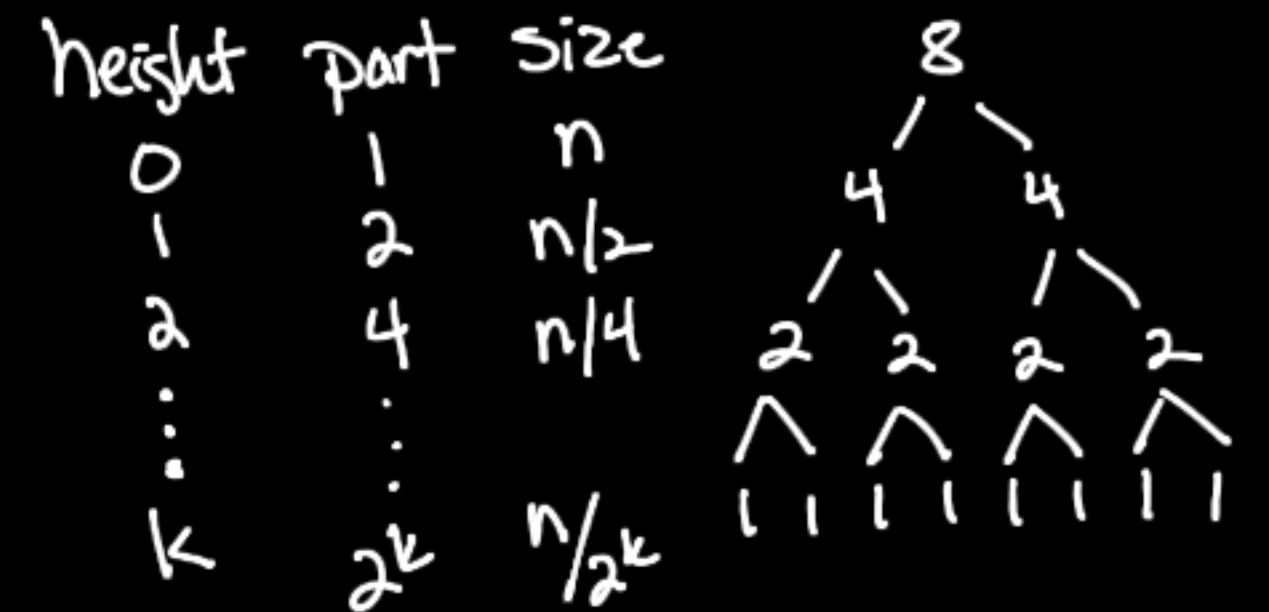
merge() is $O(n)$ at each level

each recursive call creates 2 partitions

total # of partitions depends on n

thus the height depends on n

recursion continues until one item



if n is a power of 2

height = k and we can use it to find k

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \log_2 n = k$$

merge is $O(n)$ and occurs k times

therefore $O(nk) = O(n \log_2 n)$

if n is not a power of 2

$$k = \lceil \log_2 n \rceil$$

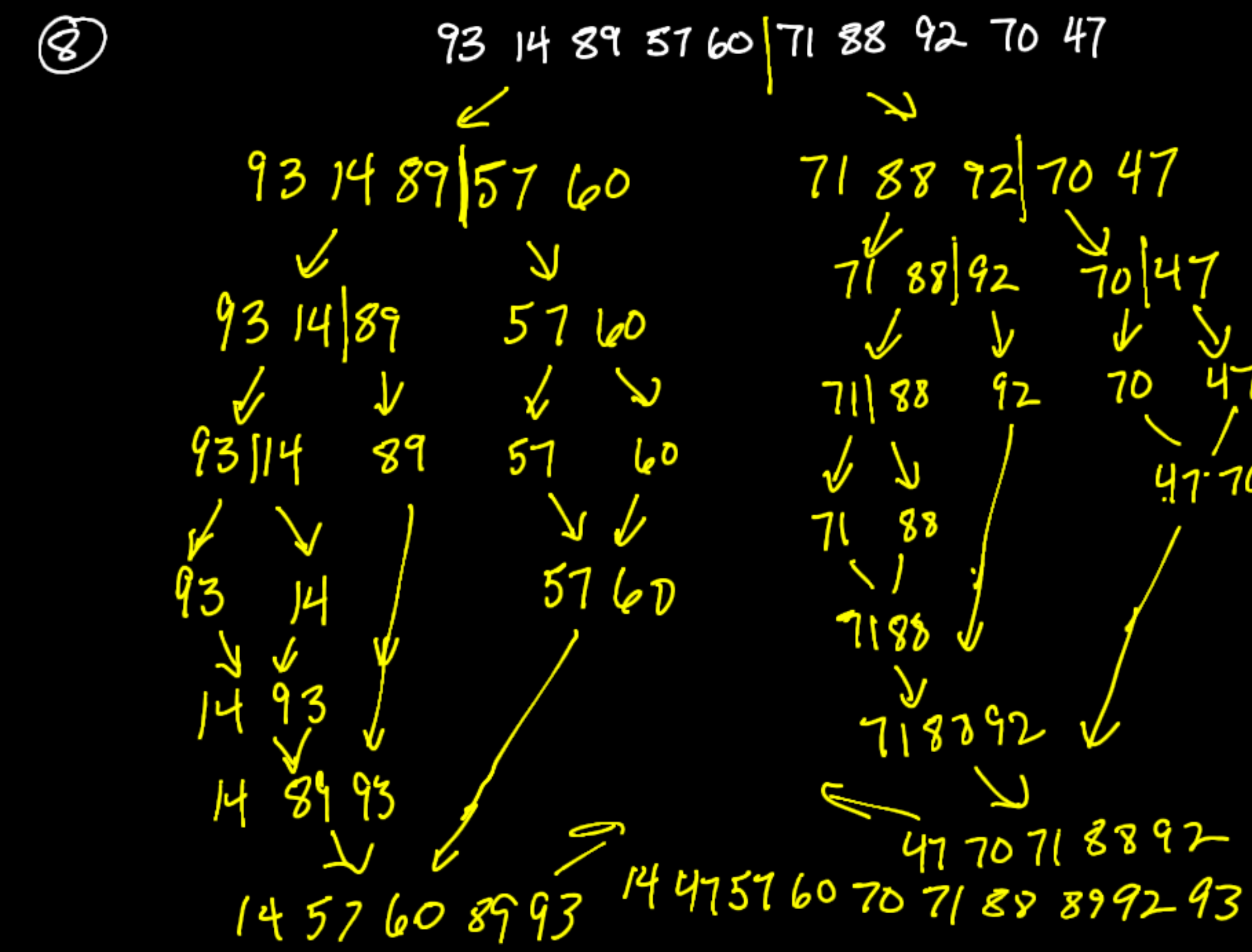
$$O(nk) = O(n(1 + \log_2 n))$$

$$= O(n + n \log_2 n)$$

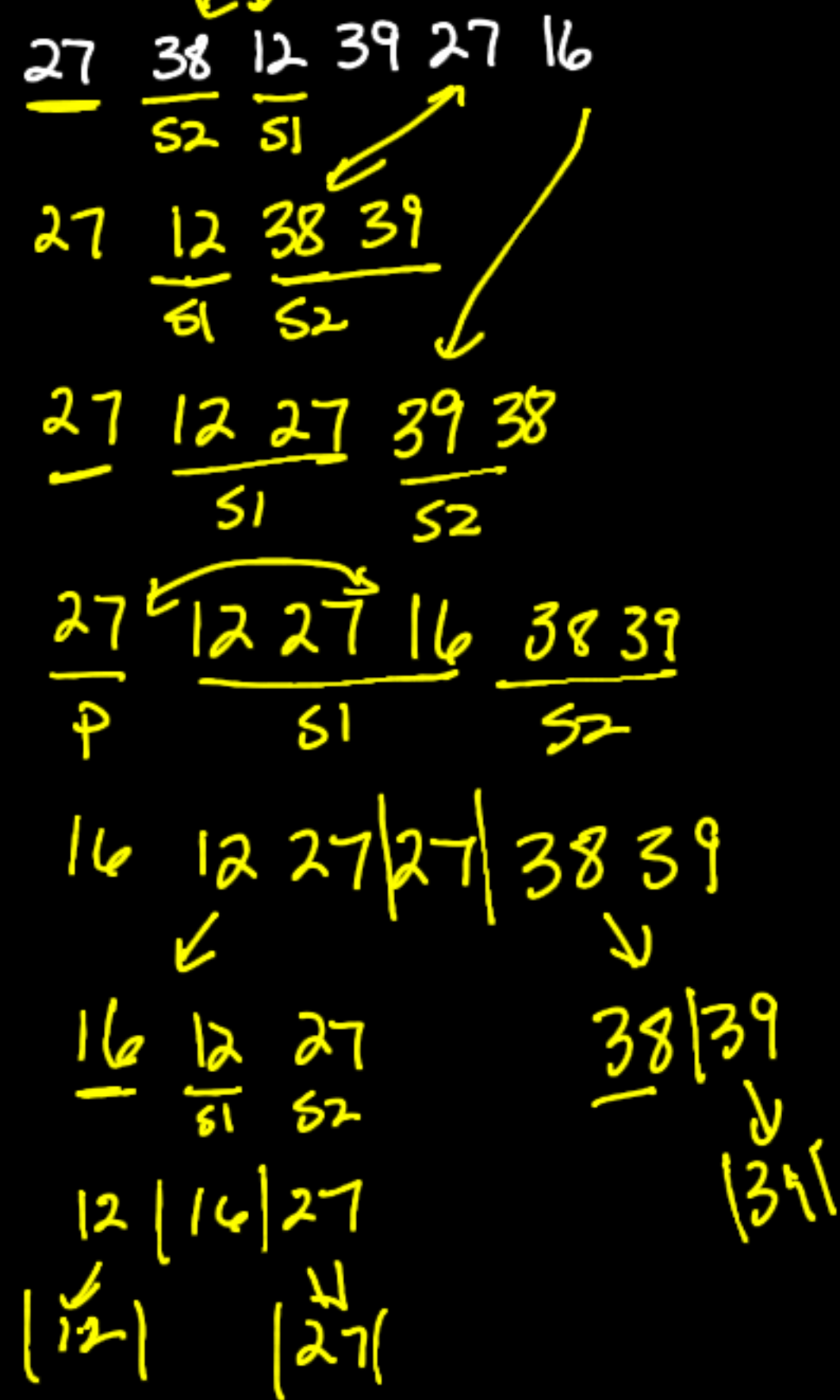
$$= O(n \log_2 n)$$

not a full level

Note: Big O allows us to drop the floor and n factors



⑨ Quick sort



→ ⑩ Analysis

mergesort is $O(n \log n)$ where n is the linear merge and $\log n$ is the recursion
 quicksort is $O(n \log n)$

- n is the linear scanning into $S1$ and $S2$
- $\log n$ is the recursion

however mergesort is always $O(n \log n)$

best case - already in order

Average case - some in order, some not

Worst case - reversed order

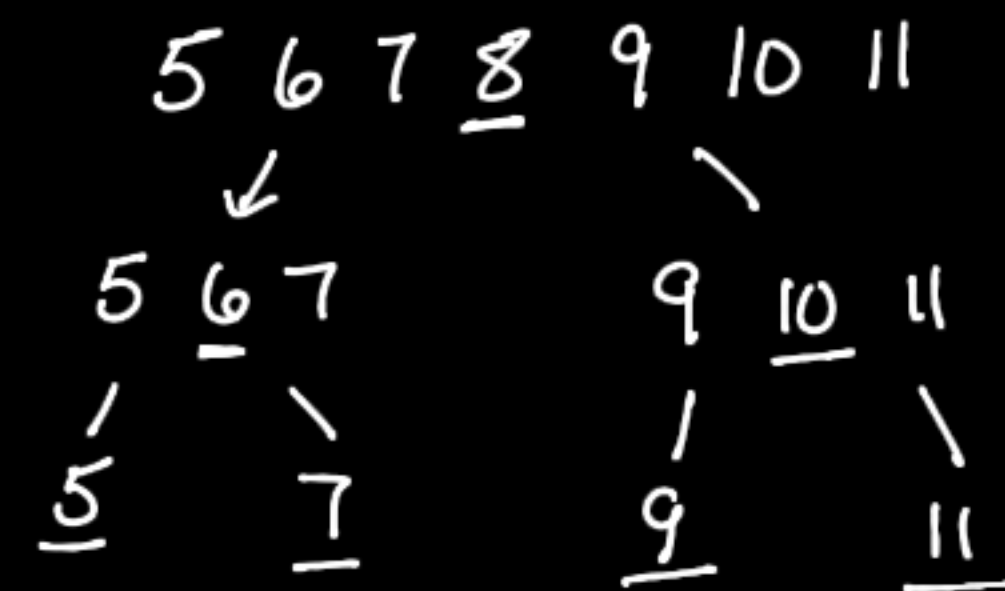
But is this true for quick sort?

best case - already in order?

5 6 7 8 9
9 8 7 6 5



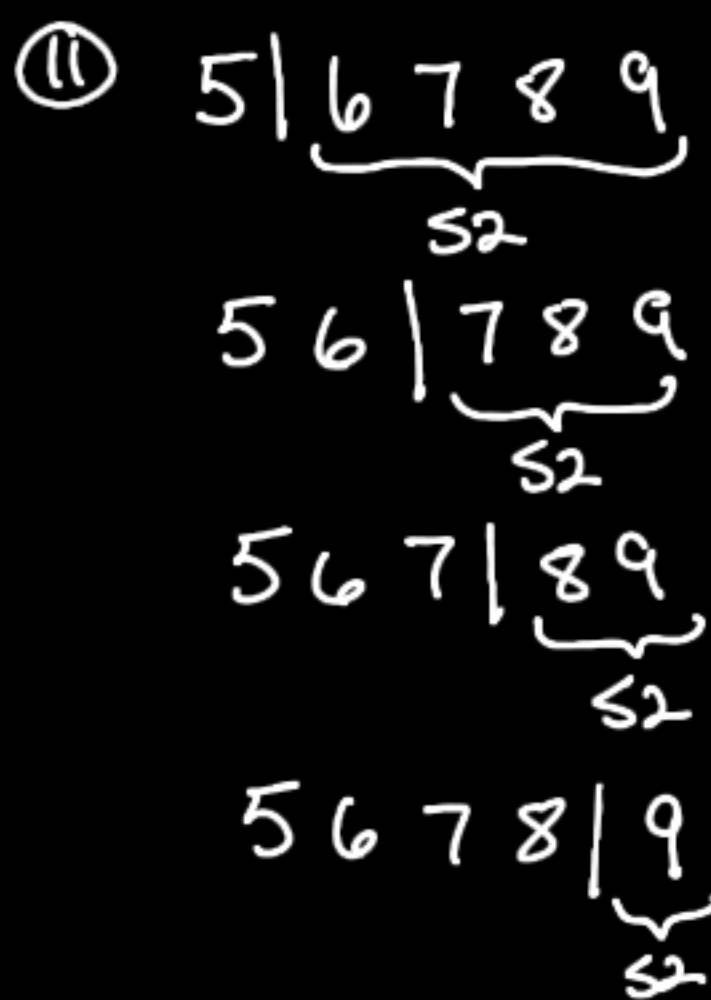
→ ⑪ Pivot selection



Best case - $O(n \log n)$
 Average case - $O(n \log n)$
 Worst case - $O(n^2)$

Common pivot selection

- random
- middle index item
- median of first, middle, and last index items



Pivot never moves

Scanning to form $S2$ is linear

recursion only on $S2$

no partitioning

Linear recursion

Scanning occurs n times (once per call)

therefore $O(n \cdot n) = O(n^2)$

But is this the worst case?

- depends on the height

- depends on the balancing of partition size

Therefore, what determines the performance?

⑬ median
54

20 26 93 17 77 31 44 55 54
54 | 26 93 17 77 31 44 55 20
 s1 s2

54 | 26 17 93 77
 s1 s2

54 | 26 17 31 77 93
 s1 s2

54 | 26 17 31 44 93 77
 s1 s2

54 | 26 17 31 44 93 77 55
 s1 s2

54 | 26 17 31 44 20 77 55 93
 s1 s2

20 26 17 31 44 | 54 | 77 55 93
 ↙ ↘